Xpert Cold Start: Dossier-ARR-PRISM

Focus and approach We prioritize people and organisation context, open-loop discovery, conservative closure, and restraint in assistant interventions. A false completion hides work; an intrusive reminder can make an otherwise useful assistant unwelcome. We preserve the existing Python architecture and use selective model inference rather than an expensive multi-agent system. The submission is a batch JSON program, with no frontend or database.

Dossiers combine saved-name aliases, seed relationships, chat membership and short speaker-attributed message samples. Organisation routing separates home and work, uses member overlap, and guards against treating external client businesses as owned organisations. Goals require message evidence and user endorsement, with at most ten.

Anchor-Retrieve-Reconcile keeps original turns and searches chronological local and cross-chat trails for each commitment, request, deadline or decision. Dates anchor to message time. Passing a due date never means completion; acknowledgements, future intent, partial progress and unrelated speakers cannot establish execution. Explicit cancellation can drop a loop; later failure evidence can reopen it. We enumerate all history: only the official checker knows the hidden scoring cutoff.

PRISM uses an asymmetric loss gate for assist/remind/leave. These loss values and probabilities are hand-set design choices, not empirically calibrated posteriors. External actions require approval; private preparation can proceed without it. Corrections override individual relationships, chat owners and task owners/statuses. Closing a corrected task requires actual later message evidence.

Evaluation and cost On 7 November 2026 we downloaded the official starter and dev archives directly from the challenge. Schema, checker and example match after newline normalization. Dev has 4,502 messages and 138 chats. Goals and interventions are not labelled there, so its denominator is 56, whereas the worked example scores all seven categories /74.

Deterministic run	Official example /74	Official dev /56	Calls / tokens
Original Arena	74.00	18.68	0 / 0
Hardened implementation	74.00	19.61	0 / 0

The original dev run took 7.00 seconds; the hardened run took 8.59 seconds on Windows. Both cost $0 per 1,000 messages because neither called a model. Real-model quality, provider token counts and network runtime remain unverified until credentials are available to the executing process. No production model is selected on synthetic scores. The evaluator uses OpenRouter; the submitted identifier must be the exact ID we actually test before submission.

Historical synthetic results (92.58/93 under a custom rubric) used a test-aware mock responder with estimated tokens. They establish regression behavior only. They are not official dev scores, live-model measurements, hidden-set estimates or rankings. The 20,000-message/320-chat synthetic tests likewise measure local compute with sparse tasks; they do not establish a 30-minute live-model bound.

Where it fails In genuine dev evaluation, loop precision improved from 7.25% to 7.84%, recall from 37.84% to 43.24%, and F1 from 12.17% to 13.28%. There were 32 matched loops among 408 predictions against 74 labels, one false DONE, 16 missed completions among matched loops and five incorrect task owners. The official dev README warns that loop labels are incomplete; unmatched predictions are not all necessarily invalid tasks.

Unseeded circle accuracy is 53.66%. Sparse contacts, dual roles, family discussions containing work vocabulary, implicit obligations, short unquoted closures and missing cross-chat topic links remain weak.

Some labels accept extremely indirect messages. We do not hardcode their identities or duplicate tasks to exploit repeated answer sources. Dev goals and intervention errors cannot be measured, and hidden performance is unknown. Better live-model coverage must be demonstrated rather than assumed.

Engineering decisions Inference never reads answer files. The client sends only to the configured model endpoint, caches repeated requests, limits concurrency, rechecks deadlines on retries, and reserves estimated in-flight request cost. Provider-reported cost, when supplied, is recorded separately from price-table estimates. Estimates cannot guarantee billing for failures where a provider omits usage. Invalid output fails with a nonzero exit; validated output is replaced atomically. Offline CI covers clean dependency installation, regressions, official example, schema/references and paths with spaces. Private datasets, runtime output and credentials are ignored. Remaining live-model and Linux checks are reported honestly in EVALUATION.md; final submission requires user approval.